

Arquitectura de Big Data Analytics por Capas para la Detección de Audio Deepfake con wav2vec 2.0 y Apache Spark

Milagros Valverde Avila
Universidad del Pacífico
Lima, Perú

mm.valverdea@alum.up.edu.pe

Sebastian Minchola Robles
Universidad del Pacífico
Lima, Perú

ws.mincholar@alum.up.edu.pe

Sebastian Guevara Peralta
Universidad del Pacífico
Lima, Perú

sa.guevarap@alum.up.edu.pe

Fiorella Tamariz Pantoja
Universidad del Pacífico
Lima, Perú

fa.tamarizp@alum.up.edu.pe

Margot Torre Solis
Universidad del Pacífico
Lima, Perú
sm.torres@alum.up.edu.pe

Resumen—La detección de audio sintético no depende únicamente del algoritmo de clasificación: exige integrar fuentes heterogéneas, administrar representaciones de alta dimensionalidad, controlar la calidad y conservar evidencia reproducible. Este trabajo diseña e implementa un prototipo de Big Data Analytics organizado en cuatro capas: Workload, Landing, Curated y Functional. La solución combina dos archivos Parquet derivados de ASVspoof 2021 con 1,005 audios reales procedentes de videos de YouTube, genera embeddings wav2vec 2.0 de 768 dimensiones y utiliza Apache Spark para persistencia, validación, integración, preparación y modelado. El train original contenía 550,185 observaciones; después del enriquecimiento alcanzó 551,190 y, tras undersampling, produjo 51,069 registros aproximadamente balanceados. Una ejecución completa registró 35.23 minutos y mostró que la conversión de audio a embeddings fue la operación dominante. Sobre un test no balanceado de 138,105 observaciones se compararon Regresión Logística, Random Forest y GBT. La Regresión Logística obtuvo ROC-AUC de 0.9230, F1 ponderado de 0.8826, Macro-F1 de 0.6151 y balanced accuracy de 0.8385. La diferencia entre el F1 ponderado y el F1 REAL de 0.3209 evidencia que las métricas agregadas deben complementarse con resultados por clase. La contribución es un flujo trazable desde archivos de origen hasta datos listos para Machine Learning y artefactos de evaluación, junto con una delimitación explícita de sus alcances de escalabilidad y generalización.

Index Terms—Big Data Analytics, audio deepfake, Apache Spark, wav2vec 2.0, arquitectura por capas, calidad de datos, clases desbalanceadas

I. INTRODUCCIÓN

La síntesis y clonación de voz han reducido el costo técnico de producir audio artificial convincente. Aunque estas tecnologías permiten aplicaciones legítimas, también pueden facilitar suplantación de identidad, fraude e ingeniería social [1], [2]. La Comisión Federal de Comercio de Estados Unidos ha advertido sobre fraudes que aprovechan muestras breves de voz [3]. Asimismo, se han documentado incidentes en los que audio e imagen generados por inteligencia artificial fueron utilizados para inducir transferencias o suplantar autoridades [4], [5]. Estos escenarios convierten la detección de audio deepfake

en un problema que involucra tanto capacidad predictiva como administración confiable de datos.

La literatura suele concentrarse en representaciones acústicas, arquitecturas neuronales y exactitud de clasificación. Sin embargo, una implementación aplicada debe resolver además la ingesta de archivos, la transformación de señal cruda, el almacenamiento de vectores de cientos de variables, la incorporación de nuevas fuentes, el control de calidad y la generación reproducible de resultados. Cuando estas actividades se tratan como fragmentos aislados, aparecen inconsistencias entre unidades de análisis, tablas sobrescritas, métricas mal interpretadas y procesos difíciles de repetir. Por ello, este proyecto aborda la detección como un flujo de Big Data Analytics en el que Machine Learning consume una estructura de datos previamente organizada.

La pregunta de investigación es: *¿cómo organizar una arquitectura por capas que integre audio crudo y embeddings precomputados, aplique controles de calidad, prepare datos con Spark y permita evaluar clasificadores de audio deepfake de forma trazable?* El objetivo general es diseñar, implementar y evaluar esa arquitectura utilizando Parquet, wav2vec 2.0, tablas persistentes y Spark ML. Los objetivos específicos son: i) conservar y describir las fuentes en Workload; ii) persistir embeddings externos y procedencia en Landing; iii) validar tipos, completitud y unicidad en Curated; iv) integrar y ensamblar las características en Functional; v) registrar volumen y tiempo por etapa; y vi) evaluar modelos con métricas adecuadas para un test desbalanceado.

Las contribuciones son cuatro. Primero, se establece una correspondencia explícita entre cada capa, su operación y su artefacto. Segundo, se incorpora una fuente externa REAL mediante una transformación reinicializable de WAV a embeddings. Tercero, se documentan controles de esquema, nulos, infinitos, duplicados y procedencia. Cuarto, se analiza el costo temporal y el desempeño predictivo sin confundir el uso de Spark con una demostración automática de escalabilidad horizontal.

El resto del artículo se organiza de la siguiente manera. La Sección II revisa detección deepfake, representaciones preentrenadas, procesamiento distribuido y evaluación desbalanceada. La Sección III define requisitos y unidades de análisis. La Sección IV describe la metodología por capas. La Sección V presenta resultados de infraestructura, calidad, tiempo y modelos. Las Secciones VI y VII discuten los hallazgos, amenazas a la validez y conclusiones.

II. ESTADO DEL ARTE

II-A. Detección de audio deepfake

ASVspoof 2021 incorporó una tarea específica de detección de voz deepfake y publicó datos, protocolos y líneas base para comparar contramedidas [6], [7]. El análisis posterior mostró que compresión, canal y cambio de dominio siguen afectando la generalización [8]. Estas limitaciones son relevantes porque un clasificador puede aprender características vinculadas con el corpus, el códec o la fuente, en lugar de señales inherentes a la generación sintética.

Li et al. construyeron un escenario de dominio cruzado con ataques generados por modelos recientes y observaron sensibilidad frente a compresores y dominios no vistos [9]. Phukan et al. estudiaron modelos preentrenados monolingües y multilingües, destacando la importancia de la diversidad del preentrenamiento [10]. En conjunto, estos trabajos muestran que un buen resultado dentro de un corpus no garantiza desempeño frente a condiciones externas. En este estudio, la combinación de ASVspoof con voz REAL de YouTube aumenta heterogeneidad, pero también introduce un posible cambio de dominio que debe declararse como limitación.

II-B. Representaciones wav2vec 2.0

wav2vec 2.0 aprende representaciones de voz mediante preentrenamiento autosupervisado sobre señal cruda [11]. El modelo transforma ventanas de audio en secuencias de estados ocultos; al aplicar promedio temporal se obtiene un vector de tamaño fijo. Kang et al. analizaron la selección y el ajuste de capas wav2vec para detección de spoofing con distintos clasificadores [12]. A diferencia de una estrategia de ajuste completo, el presente trabajo utiliza *facebook/wav2vec2-base* como extractor fijo y almacena vectores de 768 dimensiones. Esto desacopla la transformación acústica de la evaluación tabular y permite que los registros externos compartan el esquema de los Parquet existentes.

El desacoplamiento tiene ventajas operativas: los embeddings pueden persistirse, validarse y reutilizarse sin recalcular la señal en cada entrenamiento. También tiene costos. La inferencia del modelo preentrenado es la etapa más pesada sobre los WAV y, en la ejecución registrada, se realizó secuencialmente en el driver. Por tanto, la persistencia reduce recomputación posterior, pero no convierte por sí sola la extracción en un proceso distribuido.

II-C. Procesamiento analítico con Spark

MLlib integra preparación de características, algoritmos y evaluación dentro del ecosistema distribuido de Spark [13].

Tabla I
RELACIÓN CON TRABAJOS PREVIOS.

Trabajo	Énfasis	Diferencia del presente estudio
ASVspoof [6]	Benchmark y protocolos de contramedidas.	Se usa como fuente dentro de un flujo por capas.
Kang et al. [12]	Capas wav2vec y backends.	Se priorizan persistencia, calidad e integración.
Li et al. [9]	Generalización entre dominios.	Se documenta el riesgo ASVspoof-YouTube.
Meng et al. [13]	ML distribuido en Spark.	Se aplica Spark dentro de una arquitectura completa.

El uso de formatos columnares y tablas persistentes permite seleccionar variables, aplicar transformaciones y reutilizar resultados intermedios. Las arquitecturas lakehouse buscan combinar almacenamiento abierto con operaciones de ingeniería, analítica y aprendizaje automático [14]. El presente estudio adopta esa lógica de refinamiento, pero utiliza la nomenclatura del curso: Workload, Landing, Curated y Functional.

La existencia de un motor distribuido no es evidencia suficiente de escalabilidad. Para sostener una afirmación de aceleración se requieren mediciones repetidas, recursos conocidos, particionamiento controlado y comparación contra una línea base. Free Edition ofrece únicamente cómputo serverless de tamaño y uso limitados, sin configuración personalizada ni GPU [15]. En consecuencia, este artículo reporta el entorno lógico y los tiempos observados, pero no atribuye los resultados a una configuración física que la plataforma no expone.

II-D. Clases desbalanceadas y evaluación

El aprendizaje con clases desbalanceadas requiere estrategias que no oculten la clase minoritaria [16]. Accuracy, precisión ponderada y F1 ponderado pueden estar dominados por la clase con mayor prevalencia. Las curvas Precision-Recall y las métricas por clase aportan una visión complementaria a ROC-AUC [17]. Para ASVspoof, el protocolo oficial utiliza scores por archivo y EER, además de restricciones sobre datos externos [18]. Este proyecto emplea ASVspoof como fuente, pero no reproduce una participación oficial: sus observaciones son embeddings tabulares, incorpora voz externa y estima EER por fila.

III. REQUISITOS Y UNIDADES DE ANÁLISIS

III-A. Fuentes y alcance del corpus

La primera fuente es *ASVspoof2021_train_wav2vec.parquet*, con 550,185 filas. La segunda es *ASVspoof2021_test_wav2vec.parquet*, con 138,105 filas. Ambos archivos contienen *emb_0-emb_767* y *label*. La tercera fuente es *Segmento_Audio.zip*, que contiene 1,006 WAV derivados de videos de YouTube sobre temas de física; en concreto, fueron 5 videos de 2 horas aproximadamente cada uno, en inglés.

III-B. Archivo, segmento, fila y variable

La unidad de archivo corresponde a un WAV. La función de segmentación utiliza ventanas de dos segundos y desplaza-

Tabla II
FUENTES, VOLUMEN Y ESQUEMA.

Fuente	Formato	Cantidad	Unidad
Train ASVspoof	Parquet	550,185	filas de 768D
Test ASVspoof	Parquet	138,105	filas de 768D
Audio externo	WAV	1,006	archivos de- tectados
Audio válido	tabla	1,005	segmentos/filas

miento de un segundo. En principio, un archivo puede generar múltiples segmentos; en la ejecución persistida, cada uno de los 1,005 archivos válidos generó una sola fila. La unidad tabular es, por tanto, un segmento representado mediante 768 variables y una etiqueta.

La etiqueta REAL se asignó a los registros externos provenientes de YouTube. En los modelos, REAL fue codificada como 0 y FAKE como 1. La procedencia `source_file` se mantuvo en Landing y Curated, pero se excluyó de las variables predictivas. Esta decisión conserva trazabilidad sin permitir que el nombre del archivo actúe como característica.

III-C. Requisitos funcionales y de calidad

La arquitectura debía satisfacer seis requisitos. Primero, preservar los archivos originales y evitar su modificación durante el análisis. Segundo, permitir reanudación del procesamiento de WAV sin duplicar registros ya persistidos. Tercero, comprobar que cada embedding tuviera 768 dimensiones y tipos compatibles. Cuarto, detectar valores nulos, NaN, infinitos y duplicados exactos. Quinto, integrar los datos externos sin sobrescribir el train base. Sexto, producir una salida Funcional consumible por los tres modelos bajo el mismo esquema.

IV. METODOLOGÍA

IV-A. Diseño general de la arquitectura

La Fig. 1 muestra las cuatro capas. Workload concentra fuentes; Landing recibe los embeddings crudos y la procedencia; Curated aplica controles y homologa tipos; Funcional integra y construye *features*. Los modelos aparecen fuera de las capas como consumidores de la salida Funcional. Esta separación impide describir Machine Learning como si reemplazara al proceso de gestión de datos.

IV-B. Capa Workload

Workload contiene los Parquet y el ZIP original dentro de un Volume. El procedimiento verifica si el directorio de extracción ya existe antes de descomprimir, lo que evita repetir una operación innecesaria. Luego descubre los WAV de forma recursiva y ordena sus rutas. Esta capa conserva los datos tal como fueron recibidos y establece los identificadores que se utilizarán para determinar qué archivos ya fueron procesados.

Los Parquet se cargan como tablas Spark sin transformarlos con pandas. Esta decisión es importante porque el train contiene cientos de millones de valores numéricos. La lectura columnar permite trabajar con el esquema completo y seleccionar variables cuando se requieran. Workload no aplica limpieza ni balanceo; su responsabilidad es disponibilidad y preservación.

IV-C. Transición Workload–Landing

La transformación acústica se ejecuta antes de persistir Landing. Cada WAV se lee con `soundfile`, se convierte a tensor, se promedian canales cuando la señal no es mono y se remuestrea a 16 kHz. Los archivos con menos de dos segundos se omiten. Para cada ventana válida, *facebook/wav2vec2-base* genera estados ocultos; el promedio temporal produce un vector de 768 dimensiones.

El modelo `wav2vec` se carga una sola vez. La ejecución utiliza `torch.no_grad()` para evitar gradientes, dado que no se ajustan parámetros. La extracción fue secuencial en el driver y no empleó UDF, particiones Spark ni inferencia distribuida. Esta precisión es central para interpretar los tiempos: aunque el resultado se integra a un pipeline Spark, la operación más costosa no fue distribuida.

El procesamiento es reinicializable. Si `audio_ext_landing` existe, se recolectan los nombres distintos ya persistidos y se procesan únicamente los pendientes. Los resultados se acumulan en lotes de hasta 50 archivos y se escriben en modo `append`. Esta estrategia reduce la pérdida de avance ante interrupciones y explica por qué la tabla final conserva 1,005 registros válidos aun cuando se detectaron 1,006 WAV (uno presentó un error, por eso se excluyó).

IV-D. Capa Landing

Landing almacena las 768 variables, la etiqueta REAL y `source_file`. Los embeddings se conservan antes de las transformaciones de calidad de Curated. El nombre del archivo cumple dos funciones: permite reanudar el proceso sin duplicar fuentes y habilita controles de número de segmentos por archivo. En esta ejecución, la cardinalidad de `source_file` fue igual al número de filas, por lo que se obtuvo una ventana persistida por archivo válido.

La capa representa un punto de desacoplamiento. Si se modifica el balanceo o el clasificador, no es necesario volver a ejecutar `wav2vec`. Si cambian la frecuencia, la ventana, el modelo preentrenado o la regla de pooling, Landing debe regenerarse porque esos parámetros definen la representación almacenada.

IV-E. Capa Curated

Curated verifica primero que existan exactamente 768 columnas con prefijo `emb_`. Después calcula nulos y NaN por variable, valores infinitos, fuentes con más de un segmento y filas de embedding exactamente duplicadas. El control de duplicados agrupa las 768 variables y la etiqueta; `source_file` no se utiliza para ocultar repeticiones de contenido con nombres diferentes.

Las variables se convierten a `DOUBLE`. Si se detectan valores inválidos, se construye una condición conjunta para excluir las filas afectadas. La procedencia se mantiene, mientras que la selección final para integración conserva únicamente las columnas del train original. El resultado se escribe como `audio_ext_curated`. Así se separa la evidencia de calidad del esquema utilizado por los modelos.

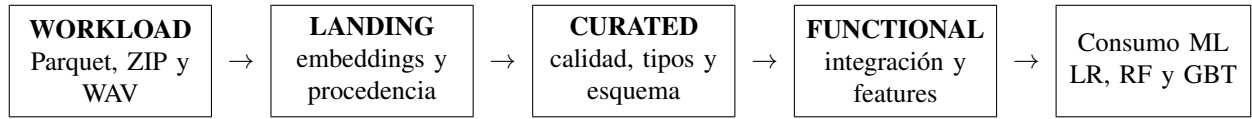


Figura 1. Arquitectura implementada. Functional constituye la última capa y deja los datos listos para el consumo posterior mediante Machine Learning.

Tabla III
CONTROLES APLICADOS EN CURATED.

Control	Resultado
Dimensiones del embedding	768
Nulos o NaN	0
Valores infinitos	0
Duplicados exactos	0
Fuentes con múltiples segmentos	0
Archivos fuente únicos	1,005
Filas persistidas	1,005
Compatibilidad con train	validada

Tabla IV
ARTEFACTOS PRINCIPALES POR CAPA.

Capa	Artefactos y responsabilidad
Workload	Parquet, ZIP y WAV originales; descubrimiento de fuentes.
Landing	audio_ext_landing; embeddings, etiqueta y procedencia.
Curated	audio_ext_curated; calidad, tipos y esquema homogéneo.
Functional	deepfake_train_enriched y tablas features.

IV-F. Capa Functional

Functional selecciona de Curated las mismas columnas que posee deepfake_train. La igualdad de esquemas se verifica antes de unionByName. El train base se cuenta antes de la unión y no se sobrescribe. El resultado se guarda como deepfake_train_enriched, y una aserción comprueba que el total sea igual a original más externos. Esta secuencia corrige el riesgo de evaluación diferida que anteriormente produjo conteos inconsistentes al sobrescribir la tabla de origen.

Luego se identifican las 768 columnas y VectorAssembler las combina en features. handleInvalid=error impide que una observación inválida avance silenciosamente. El train y el test transformados se escriben en tablas separadas. Con ello termina la arquitectura de datos: Functional entrega pares features-label listos para cualquier consumidor compatible con Spark ML.

IV-G. Balanceo y consumo mediante ML

Después de Functional, las etiquetas se codifican como REAL=0 y FAKE=1. El train enriquecido contiene 525,677 FAKE y 25,513 REAL. Se aplica undersampling aleatorio a FAKE con fracción 0.048534 y semilla 42. El resultado contiene 25,556 FAKE y 25,513 REAL. El test se conserva sin balancear para aproximar la distribución del corpus disponible.

El undersampling reduce costo y evita replicar observaciones REAL, pero descarta 95.14% de FAKE. Se prefirió

Tabla V
CONFIGURACIÓN DE LOS MODELOS.

Componente	Configuración
Entrada	768 variables en features.
Muestreo	fracción 0.048534; semilla 42.
LR	100 iter.; regularización 0; umbral 0.5.
RF	100 árboles; profundidad 10.
GBT	50 iter.; profundidad 5.

frente a oversampling simple para no multiplicar ejemplos y frente a SMOTE porque los puntos sintéticos en un espacio de embedding no tienen una correspondencia verificable con audio válido. Los pesos de clase conservarían información, pero exigirían entrenar con el volumen completo y ajustar el tratamiento por algoritmo. Estas alternativas se consideran comparaciones futuras, no evidencia ejecutada.

Se entrenaron Regresión Logística [19], Random Forest [20] y GBT [21]. LR utilizó maxIter=100, regParam=0.0, elasticNetParam=0.0 y umbral 0.5. RF utilizó 100 árboles y profundidad 10. GBT empleó 50 iteraciones y profundidad 5. Las semillas de RF y GBT no se fijaron explícitamente, por lo que una nueva ejecución puede variar.

IV-H. Métricas y protocolo de evaluación

Se calcularon accuracy, precisión, recall y F1 ponderados; ROC-AUC; PR-AUC para FAKE; precisión, recall y F1 por clase; Macro-F1; balanced accuracy; y EER aproximado. La matriz usa REAL como clase 0 y FAKE como clase 1. Un falso negativo FAKE corresponde a audio sintético clasificado como REAL. El EER se obtiene buscando el punto donde FPR y FNR son más cercanos sobre scores por observación.

El PR-AUC de FAKE debe interpretarse considerando que FAKE representa 95.48% del test. Una curva PR elevada es esperable cuando la clase positiva domina. Por ello, la selección no se fundamenta únicamente en PR-AUC, sino en la combinación de ROC-AUC, Macro-F1, balanced accuracy y resultados por clase. Los tres modelos se comparan sobre el mismo test; en consecuencia, la elección del mejor debe considerarse parte del análisis comparativo y no una estimación independiente después de selección.

IV-I. Medición temporal y entorno

Las celdas instrumentadas usan time.perf_counter(). Para evitar medir solo la construcción perezosa de un DataFrame, se ejecutan acciones como count(), fit(), evaluación o escritura. No se emplean cache() ni persist(). La medición corresponde a una corrida y no a un benchmark repetido.

El entorno reportó Python 3.12.3, Spark/PySpark 4.1.0, pandas 2.2.3 y spark.sql.shuffle.partitions=auto.

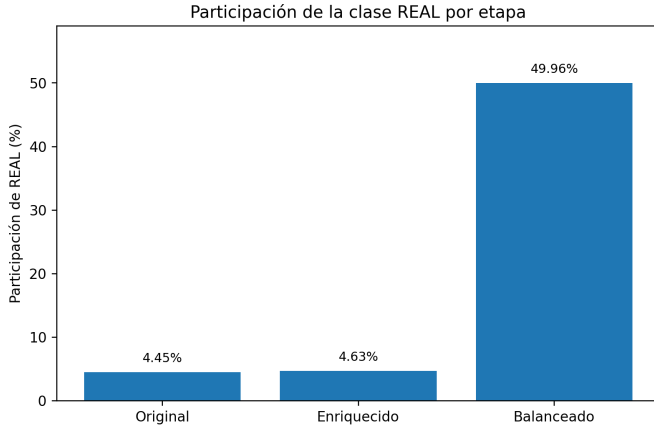


Figura 2. Participación de REAL en el train original, enriquecido y balanceado.

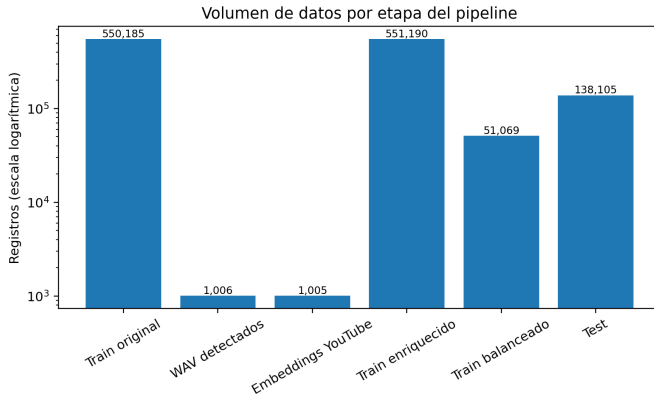


Figura 3. Volumen de registros por etapa; la escala logarítmica permite comparar magnitudes distintas.

La plataforma fue Databricks Serverless/Spark Connect sobre Linux ARM64. Java, Spark master, paralelismo y recursos físicos no fueron accesibles. Por ello, CPU y RAM no se infieren ni se atribuyen a una máquina virtual específica.

V. RESULTADOS

V-A. Volumen y evolución del pipeline

El train pasó de 550,185 a 551,190 filas tras incorporar 1,005 embeddings REAL. La clase REAL aumentó de 24,508 a 25,513, equivalente a 4.10 % respecto de su cantidad original. Sin embargo, su participación solo cambió de 4.45 % a 4.63 %. El enriquecimiento aumenta diversidad de fuentes, pero no resuelve el desbalance por sí mismo.

Después del undersampling, se conservaron 25,556 FAKE y las 25,513 REAL. El train balanceado alcanzó 51,069 observaciones y una participación REAL de 49.96 %. El test mantuvo 131,860 FAKE y 6,245 REAL. La Fig. 2 muestra que el principal cambio de proporción proviene del muestreo, no del enriquecimiento.

La Fig. 3 evidencia que las capas no deben compararse solo por número de filas. Landing y Curated contienen 1,005

registros externos, mientras Functional integra más de medio millón de filas y 768 variables. El undersampling disminuye el número de observaciones que llega al entrenamiento, pero la preparación previa debe procesar el esquema completo.

V-B. Calidad y trazabilidad

La validación Curated confirmó 768 dimensiones, cero nulos o NaN, cero infinitos y cero duplicados exactos. También confirmó 1,005 archivos fuente únicos y ninguna fuente con múltiples segmentos persistidos. La igualdad entre fuentes y filas no se generaliza a otros audios: es una propiedad de esta ejecución y de las duraciones procesadas.

La integración verificó 769 columnas compatibles, correspondientes a 768 embeddings y una etiqueta. `source_file` se mantuvo en las capas donde aporta procedencia y se retiró al seleccionar el esquema de entrenamiento. Los conteos antes y después de la unión cumplieron $550,185 + 1,005 = 551,190$, evitando la inconsistencia de versiones previas.

V-C. Tiempos de ejecución

La ejecución completa registró 35.23 minutos. La Tabla VI combina los tiempos impresos por las celdas con el resumen consolidado. La fila de operaciones auxiliares es la diferencia de 2.95 minutos entre el total y las etapas individualizadas; incluye visualizaciones, materializaciones y coordinación no asignadas a una sola capa.

La conversión a embeddings consumió 13.75 minutos, 39.03 % del total, y fue el mayor cuello de botella individual. Entre los clasificadores, GBT tardó 4.95 minutos, seguido de RF con 2.35 y LR con 1.22. La relación es coherente con la complejidad de los ensambles, aunque una sola corrida no permite separar variación de infraestructura, calentamiento y costo algorítmico.

La suma de carga y conversión wav2vec alcanzó 14.70 minutos. Esto refuerza el valor de persistir Landing: reentrenar modelos no requiere repetir esa transformación. Functional tomó 3.10 minutos para integrar y ensamblar el esquema, mientras Curated registró 0.48 minutos. Los resultados sugieren que la principal oportunidad de optimización está en paralelizar o servir la extracción, no en eliminar los controles de calidad.

V-D. Resultados predictivos

La Tabla VII muestra que LR obtuvo el mejor ROC-AUC, F1 ponderado, recall FAKE y Macro-F1. GBT logró el mayor recall REAL, pero el menor recall FAKE. Las diferencias globales entre modelos son reducidas; la brecha más importante aparece entre el desempeño de las dos clases.

Para LR, 110,756 FAKE y 5,227 REAL fueron clasificados correctamente. Se registraron 21,104 FAKE clasificados como REAL y 1,018 REAL clasificados como FAKE. El recall FAKE de 0.8400 significa que aproximadamente 16 % de los sintéticos no fueron detectados. La precisión REAL de 0.1985 indica que muchas predicciones REAL correspondieron en realidad a FAKE.

La Fig. 4 muestra ROC-AUC superior a 0.91 para los tres modelos. Las curvas PR se mantienen altas debido a que FAKE

Tabla VI
TIEMPO REGISTRADO POR ETAPA DEL PIPELINE.

Capa/ámbito	Etapa	min	Capa/ámbito	Etapa	min
Preparación	Entorno y dependencias	1.85	Funcional	Integración y ensamblaje	3.10
Workload	Lectura y descubrimiento	1.10	Consumo ML	Undersampling	0.65
Workload-Landing	Carga wav2vec	0.95	Consumo ML	Regresión Logística	1.22
Workload-Landing	Conversión a embeddings	13.75	Consumo ML	Random Forest	2.35
Curated	Calidad y persistencia	0.48	Consumo ML	GBT	4.95
Resultados	Evaluación y exportación	1.88	Transversal	Operaciones auxiliares	2.95
Tiempo total					35.23

Tabla VII
MÉTRICAS SOBRE EL CONJUNTO DE PRUEBA DESBALANCEADO.

Modelo	Accuracy	F1 pond.	ROC-AUC	PR-AUC	EER	Prec. REAL	Rec. REAL	F1 REAL	Prec. FAKE	Rec. FAKE	Macro-F1	Bal. Acc.
Regresión Logística	0.8398	0.8826	0.9230	0.9958	0.1612	0.1985	0.8370	0.3209	0.9909	0.8400	0.6151	0.8385
Random Forest	0.8324	0.8777	0.9169	0.9953	0.1697	0.1897	0.8277	0.3087	0.9903	0.8326	0.6067	0.8301
GBT	0.8289	0.8756	0.9183	0.9955	0.1633	0.1891	0.8463	0.3091	0.9913	0.8281	0.6058	0.8372

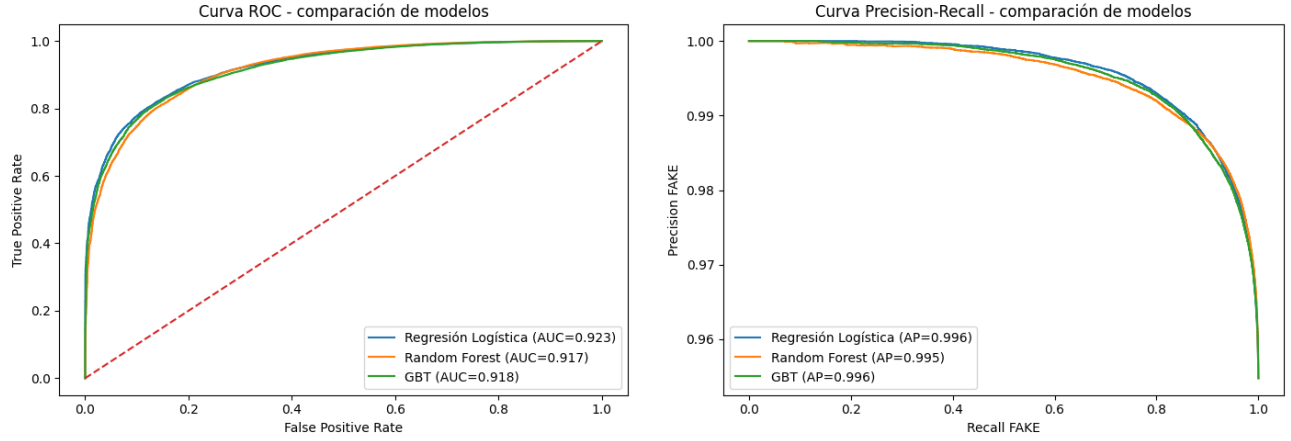


Figura 4. Curvas ROC y Precision-Recall calculadas a partir de los scores de FAKE. La elevada prevalencia de FAKE debe considerarse al interpretar PR-AUC.

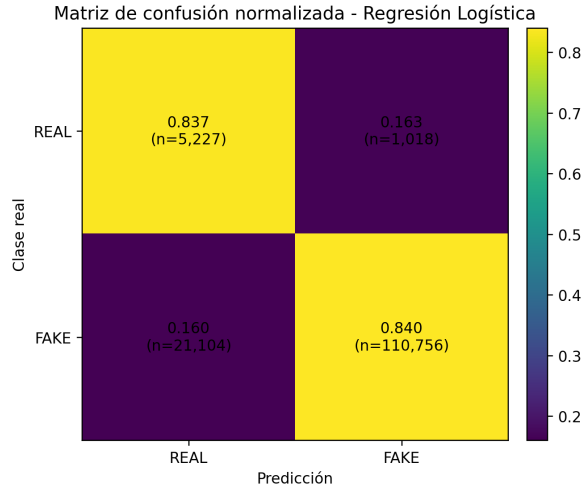


Figura 5. Matriz de confusión normalizada de Regresión Logística; se incluyen conteos absolutos.

es la clase positiva mayoritaria. La Fig. 5 permite observar simultáneamente recall REAL de 0.837 y recall FAKE de

0.840. Esta visualización evita que los conteos FAKE oculten el comportamiento sobre REAL.

El EER aproximado más bajo fue 0.1612 para LR con umbral 0.5028. GBT obtuvo 0.1633 y RF 0.1697. Estas cifras son por observación y no deben compararse directamente con el EER oficial por archivo de ASVspoof. Además, la regla que agregaría múltiples ventanas de un audio no fue validada en el test, porque las filas evaluadas ya contenían embeddings.

VI. REPRODUCIBILIDAD Y AUDITORÍA DEL PIPELINE

VI-A. Contratos de datos entre capas

Una arquitectura por capas requiere algo más que nombres consecutivos: cada transición debe definir condiciones que puedan comprobarse. En Workload, el contrato establece que los archivos de origen se encuentran disponibles, no se modifican y pueden enumerarse. En Landing, cada fila debe contener 768 valores, una etiqueta y un identificador de procedencia. En Curated, los valores deben ser finitos, los tipos compatibles y los controles de duplicidad explícitos. En Funcional, el esquema debe coincidir con el train base y la cardinalidad posterior a la unión debe ser reconciliable.

Tabla VIII
CONTRATOS E INVARIANTES DE LA ARQUITECTURA.

Capa	Entrada/transformación	Invariante de salida
Workload	Parquet, ZIP y WAV preservados.	Fuentes enumerables y sin sobrescritura.
Landing	wav2vec sobre ventanas válidas.	768D, label y source_file.
Curated	Calidad, tipado y homología.	Valores finitos y esquema validado.
Functional	Unión y VectorAssembler.	features-label listos para ML.

Estos contratos reducen acoplamiento. La función que genera embeddings no necesita conocer el algoritmo posterior; únicamente debe producir el esquema de Landing. Curated no decide el balanceo, sino que garantiza que las filas sean válidas. Functional tampoco elige el mejor clasificador: entrega una representación común. Esta separación permite sustituir un consumidor sin alterar las reglas de calidad y facilita localizar una falla en la etapa responsable.

El esquema de 768 variables funciona como contrato transversal. La aserción sobre el número de columnas se ejecuta antes de Curated y antes de VectorAssembler. Esto evita que una actualización del extractor, una selección incompleta o un cambio de nombres produzcan un modelo con entradas distintas. De manera análoga, la codificación REAL=0 y FAKE=1 se fija antes de entrenar y se utiliza para interpretar scores, matrices y falsos negativos.

VI-B. Reanudación, idempotencia y manejo de fallos

La conversión de audio fue diseñada para reanudarse porque es la etapa de mayor duración. Antes de procesar, el notebook consulta los `source_file` distintos de Landing. Un WAV ya persistido se excluye de pendientes; los nuevos resultados se escriben en lotes. En la corrida registrada, la consulta encontró 1,005 archivos persistidos y un pendiente. El último archivo no generó un segmento válido y fue reportado como omitido, sin alterar los registros anteriores.

Esta estrategia es reinicializable, aunque no completamente idempotente frente a todas las fallas posibles. Si un proceso terminara durante una escritura no atómica o si existieran nombres iguales en directorios distintos, el nombre base podría no ser una clave suficiente. Una versión más robusta debería almacenar ruta relativa, tamaño, fecha y un hash del contenido. La combinación permitiría distinguir homónimos y verificar que un archivo con el mismo nombre no haya cambiado.

La escritura de Curated y Functional utiliza `overwrite` sobre tablas derivadas, pero nunca sobre Workload ni sobre el train base. `Overwrite` es apropiado cuando la salida se reconstruye completamente desde una entrada persistida; `append` se reserva para los lotes reinicializables de Landing. Esta diferencia debe mantenerse: aplicar `append` en Curated duplicaría filas en cada corrida, mientras sobrescribir Landing durante un fallo perdería progreso.

El notebook conserva un arreglo de errores durante la transformación. Los archivos demasiado cortos y las excepciones se registran por nombre. Para una auditoría más completa, ese arreglo debería persistirse como tabla de rechazo con causa,

marca temporal, parámetros del extractor y versión del código. Así, el número de descartes podría reconciliarse mediante la relación:

$$N_{detectados} = N_{procesados} + N_{omitidos} + N_{pendientes}. \quad (1)$$

En la ejecución analizada, $1,006 = 1,005 + 1 + 0$ después de finalizar la validación del pendiente.

VI-C. Linaje y reconciliación cuantitativa

El linaje se documenta mediante nombres de tabla, procedencia y conteos. Los 1,005 registros externos pueden rastrearse desde Landing hacia Curated. Para integrarlos, se seleccionan las columnas del train base; la procedencia permanece disponible en Curated aunque no viaje al modelo. La reconciliación del train utiliza conteos materializados antes y después de la unión, con una aserción que detiene la ejecución si no se cumple la suma.

La distribución de clases ofrece un segundo control. FAKE no cambia durante el enriquecimiento, mientras REAL aumenta exactamente en 1,005. Posteriormente, el undersampling conserva todas las REAL y reduce únicamente FAKE. Estas relaciones permiten detectar una etiqueta incorrecta o una unión accidental. Las siguientes igualdades se verifican con las cifras exportadas:

$$N_{train_enr} = 525,677 + 25,513 = 551,190, \quad (2)$$

$$N_{balanceado} = 25,556 + 25,513 = 51,069. \quad (3)$$

La consistencia también se revisa entre matrices y métricas. Para LR, los cuatro conteos suman 138,105, el tamaño del test. El recall FAKE se obtiene como $110,756 / (110,756 + 21,104) = 0,8400$, y el recall REAL como $5,227 / (5,227 + 1,018) = 0,8370$. Recalcular estas tasas a partir de la matriz evita depender únicamente de nombres de columnas exportadas.

VI-D. Reproducibilidad computacional

El notebook registra versiones, hiperparámetros y tiempos. También fija la semilla del undersampling. Estas medidas permiten reconstruir gran parte del experimento, pero no garantizan repetición bit a bit: RF y GBT utilizaron semillas efectivas no fijadas; el entorno Serverless puede cambiar recursos; y la descarga del modelo puede provenir de caché o red. La reproducibilidad debe entenderse, por tanto, como capacidad de repetir el procedimiento y obtener resultados cercanos, no como identidad absoluta de cada número.

La Tabla IX resume la evidencia conservada y los elementos pendientes. El propósito no es convertir cada limitación en una falla, sino diferenciar lo demostrado de lo que requiere una nueva corrida o información adicional. Esta distinción es especialmente importante para los tiempos: el total de 35.23 minutos describe una corrida y no una distribución estadística.

Tabla IX
MATRIZ DE EVIDENCIA PARA REPRODUCIBILIDAD.

Evidencia	Estado	Artefacto
Conteos y clases por etapa	Validado	CSV/output
Calidad y esquema Curated	Validado	CSV/output
Parámetros de modelos	Registrado	notebook
Versiones de software	Registrado	notebook
Tiempo por etapa	Una corrida	notebook
Semilla de undersampling	Fijada	código
Semillas RF y GBT	No fijadas	limitación
CPU, RAM y nodos	No expuestos	limitación
Metadatos de videos/habla	Incompletos	pendiente

VI-E. Criterios para una nueva ejecución

Una nueva corrida debería comenzar desde un estado identificado. Si Landing se reutiliza, su versión de extractor debe coincidir con el código. Si se desea medir la extracción completa, es necesario escribir en una tabla nueva o vaciar deliberadamente una copia de trabajo; de lo contrario, la lógica reinicializable procesará solo pendientes. Esta diferencia explica por qué un tiempo de reanudación no es comparable con una construcción desde cero.

Para obtener tiempos más defendibles se recomiendan al menos tres repeticiones, separación entre descarga y carga en caché, y registro de hora de inicio, tamaño de entrada y conteo materializado. Deben reportarse mediana y rango, no solo el mejor valor. En los modelos, cada corrida debe fijar semillas y usar el mismo train balanceado. Si se comparan estrategias de muestreo, el costo de preparación debe medirse además del entrenamiento.

La validación del modelo debe incorporar una partición distinta del test. Los hiperparámetros y el umbral se seleccionarían en validación; el test se ejecutaría una sola vez. Para audio con múltiples ventanas, la agregación por archivo debe definirse antes de evaluar. De este modo, la unidad de decisión, la métrica y el protocolo quedarían alineados.

VII. DISCUSIÓN

VII-A. Contribución desde Big Data Analytics

El principal resultado no es que un clasificador lineal supere por un margen pequeño a dos ensambles. La contribución es la organización de fuentes heterogéneas en artefactos trazables. Workload preserva; Landing desacopla la transformación acústica; Curated hace explícita la calidad; Functional entrega un contrato tabular común. Esta organización permite repetir el consumo analítico sin recalcular audio y evita que decisiones de modelado modifiquen las fuentes originales.

El uso de Parquet y Spark resulta pertinente por la combinación de filas y dimensionalidad. Aunque 550 mil observaciones pueden parecer moderadas frente a sistemas masivos, las 768 variables producen más de 422 millones de valores solo en train. La lectura distribuida, las tablas persistentes y VectorAssembler permiten manejar esta estructura sin convertir el dataset completo a pandas. pandas se restringe a tablas agregadas, métricas y scores necesarios para curvas.

La evidencia no demuestra escalabilidad horizontal. No se registraron número de nodos, CPU, memoria, particiones efec-

tivas ni ejecución comparable fuera de Serverless. Tampoco se repitieron mediciones. La formulación correcta es que se implementó un flujo compatible con operaciones distribuidas de Spark y se identificaron costos por etapa. Este alcance es más preciso que afirmar que el sistema escala solo por usar una plataforma de Big Data.

VII-B. Bottlenecks y decisiones de persistencia

El tiempo de embeddings confirma que la frontera Workload-Landing concentra el mayor costo. La extracción en driver aprovecha PyTorch, pero no el paralelismo de Spark. Persistir el resultado es, por ello, una decisión arquitectónica y no únicamente de almacenamiento: separa un cálculo costoso del ciclo iterativo de modelos. Una ejecución futura podría distribuir archivos por partición, utilizar inferencia por lotes o invocar un servicio especializado, siempre verificando que el orden de variables y el pooling permanezcan idénticos.

GBT fue el modelo más lento, pero no mejoró las métricas principales. LR tardó aproximadamente una cuarta parte del tiempo de GBT y obtuvo mejores resultados globales. En este conjunto de embeddings, una frontera lineal parece competitiva frente a la complejidad adicional de los árboles. Sin embargo, no se realizó búsqueda sistemática de hiperparámetros, por lo que el resultado compara configuraciones concretas y no el potencial máximo de cada familia.

El undersampling disminuyó de forma drástica el costo de entrenamiento, pero eliminó 500,121 observaciones FAKE. Esto puede excluir tipos de ataque raros y hacer que el resultado dependa de la muestra elegida. Los pesos de clase permitirían conservar información; una estrategia híbrida podría reducir el volumen sin alcanzar una proporción casi 1:1. La comparación entre estas alternativas debe medir tanto desempeño como costo temporal.

VII-C. Interpretación operacional de las métricas

El F1 ponderado de 0.8826 puede parecer alto, pero el F1 REAL de 0.3209 revela una debilidad importante. La razón es la baja precisión REAL: predecir REAL no proporciona evidencia suficiente de autenticidad. En un escenario de fraude, los 21,104 falsos negativos FAKE representan el error más peligroso. En otro escenario, bloquear 1,018 audios legítimos también puede generar costos. La decisión de umbral debe responder a esos costos y no únicamente maximizar una métrica global.

La PR-AUC cercana a uno no contradice esta limitación. Cuando FAKE es positivo y representa más de 95 % del test, la precisión base ya es elevada. Macro-F1 y balanced accuracy equilibran la influencia de clases, mientras la matriz normalizada muestra tasas directamente. Reportar todas estas perspectivas reduce el riesgo de seleccionar un modelo por una cifra aislada.

VII-D. Escenarios de decisión y umbrales

La etiqueta final depende del umbral aplicado al score FAKE. El valor 0.5 utilizado por LR es una configuración estándar, pero no necesariamente minimiza el costo del problema. Si aceptar un audio sintético como REAL es el error

Tabla X
ESCENARIOS ANALÍTICOS Y CRITERIO PRIORITARIO.

Escenario	Objetivo principal	Criterio
Filtro preventivo	Reducir FAKE aceptados.	Recall FAKE y FN.
Protección de usuarios	Evitar bloqueo de voz REAL.	Recall REAL y FP.
Priorización	Ordenar revisión humana.	Curvas y calibración.
Comparación técnica	Contrastar modelos.	Macro-F1 y bal. acc.
Monitoreo	Detectar cambio temporal.	Distribución de scores.

más grave, el umbral debería disminuir para elevar recall FAKE. Esta modificación incrementaría la cantidad de audios REAL bloqueados. En cambio, un proceso orientado a evitar falsos rechazos podría aumentar el umbral y asumir más falsos negativos FAKE.

La selección puede formalizarse mediante un costo esperado. Sea t el umbral, c_{FN} el costo de aceptar FAKE y c_{FP} el costo de rechazar REAL. Una regla simplificada buscaría:

$$\min_t c_{FN}N_{FAKE}P(s < t | FAKE) + c_{FP}N_{REAL}P(s \geq t | REAL). \quad (4)$$

El experimento no dispone de valores monetarios u operativos para esos costos, por lo que no optimiza esta función. La ecuación muestra por qué EER o F1 no sustituyen una definición del uso: asignan implícitamente relaciones de costo que pueden no coincidir con el contexto.

También debe distinguirse clasificación automática de priorización. Con una precisión REAL baja, la predicción REAL no debería utilizarse como certificación de autenticidad. El score puede servir para ordenar casos y enviar una zona intermedia a revisión. Por ejemplo, scores cercanos al umbral podrían marcarse como indeterminados, mientras solo los extremos activarían una decisión automática. Este esquema requiere calibración y evaluación de cobertura, pero reduce la presión de convertir toda salida en una afirmación binaria.

VII-E. Crecimiento de datos y evolución de las capas

El diseño permite analizar qué ocurre si crece cada fuente. Un aumento de WAV afecta principalmente la transición a Landing; el costo crecería con el número y duración de ventanas. Un aumento de Parquet afecta lectura, unión y VectorAssembler. Un aumento de dimensionalidad cambiaría los contratos de Curated y Functional y obligaría a regenerar modelos. Estas formas de crecimiento son distintas y no deberían resumirse únicamente como más filas.

Si los audios futuros son más largos, dejará de cumplirse la relación una fila por archivo. Landing deberá incorporar un identificador de segmento, desplazamiento inicial, duración y posiblemente hash de ventana. Functional podrá continuar trabajando por fila, pero la evaluación necesitará una regla por archivo. El esquema actual ya anticipa esta posibilidad al separar archivo y segmento; falta materializar los metadatos temporales.

Si aumenta el número de fuentes, Workload debería evolucionar hacia un catálogo con versión, licencia, fecha de adquisición y checksum. Curated podría aplicar perfiles de calidad por origen, porque frecuencia de muestreo, códec y ruido no se distribuyen igual entre ASVspoof y YouTube. Functional debería registrar la fuente en una tabla de linaje separada, aunque no la utilice como predictor. Esto permitiría comparar desempeño por dominio sin introducir fuga de información.

La partición física también requeriría evaluación. Los Parquet podrían organizarse por conjunto o versión, pero no por cada una de las 768 variables. Un número excesivo de archivos pequeños aumentaría el costo de planificación; archivos muy grandes limitarían paralelismo. El trabajo no midió esas alternativas, de modo que cualquier recomendación concreta de tamaño sería prematura. La arquitectura identifica dónde aplicar la decisión cuando el volumen lo justifique.

VIII. CONCLUSIONES

Se diseñó e implementó una arquitectura de Big Data Analytics con cuatro capas claramente delimitadas. Workload preservó Parquet, ZIP y WAV; Landing almacenó embeddings crudos y procedencia; Curated aplicó controles de calidad y tipado; Functional integró 551,190 observaciones y construyó vectores listos para ML. Esta organización alinea texto, artefactos y figura, y permite identificar la responsabilidad de cada etapa.

La ejecución registró 35.23 minutos. La conversión a embeddings fue el mayor costo con 13.75 minutos, seguida por GBT con 4.95. Estos resultados justifican persistir representaciones y priorizar la optimización de extracción. No obstante, una sola corrida serverless no permite afirmar aceleración ni escalabilidad; se requieren repeticiones y recursos controlados.

LR presentó el mejor balance global con ROC-AUC 0.9230, Macro-F1 0.6151 y balanced accuracy 0.8385, pero el F1 REAL de 0.3209 impide interpretar el F1 ponderado como desempeño uniforme. El resultado más relevante es metodológico: evaluar clases desbalanceadas exige métricas por clase, matrices normalizadas y una lectura explícita del costo de error.

La correspondencia entre capas y contratos facilita auditar el recorrido. Los 1,006 WAV detectados se reconciliaron con 1,005 archivos válidos y un descarte; las 1,005 filas Landing se conservaron en Curated; y Functional verificó que el train enriquecido fuera igual al original más los nuevos registros. Estas relaciones son tan importantes como las métricas porque permiten identificar si una variación proviene de datos, muestreo o modelo. El uso de `source_file`, tablas derivadas separadas y aserciones de cardinalidad evita que el flujo dependa únicamente de inspecciones manuales.

Los resultados también muestran que optimizar el pipeline requiere medir la etapa correcta. La conversión wav2vec concentra más tiempo que cada modelo individual, mientras los controles Curated representan una fracción pequeña del total. Reducir validaciones produciría poco ahorro y eliminaría evidencia de calidad; en cambio, reutilizar Landing y

paralelizar la extracción ofrece una oportunidad mayor. Esta conclusión mantiene el enfoque de Big Data Analytics: rendimiento, calidad, linaje y consumo analítico deben evaluarse conjuntamente.

REFERENCIAS

- [1] M. Westerlund, "The emergence of deepfake technology: A review," *Technology Innovation Management Review*, vol. 9, no. 11, pp. 39–52, 2019.
- [2] Europol Innovation Lab, "Facing reality? law enforcement and the challenge of deepfakes," Europol, 2022, consultado en junio de 2026. [Online]. Available: <https://www.europol.europa.eu/publications-events/publications/facing-reality-law-enforcement-and-challenge-of-deepfakes>
- [3] Federal Trade Commission, "Scammers use ai to enhance their family emergency schemes," Consumer Advice, 2023, consultado en junio de 2026. [Online]. Available: <https://consumer.ftc.gov/consumer-alerts/2023/03/scammers-use-ai-enhance-their-family-emergency-schemes>
- [4] Financial Times, "Arup lost \$25mn in hong kong deepfake video conference scam," Financial Times, May 2024, consultado en junio de 2026. [Online]. Available: <https://www.ft.com/content/b977e8d4-664c-4ae4-8a8e-eb93bdf785ea>
- [5] Reuters, "Malicious actors using ai to pose as senior us officials, fbi says," Reuters, May 2025, consultado en junio de 2026. [Online]. Available: <https://www.reuters.com/world/us/malicious-actors-using-ai-pose-senior-us-officials-fbi-says-2025-05-15/>
- [6] J. Yamagishi, X. Wang, M. Todisco, M. Sahidullah, J. Patino, A. Nautsch, X. Liu, K. A. Lee, T. Kinnunen, N. Evans, and H. Delgado, "Asvspoof 2021: Accelerating progress in spoofed and deepfake speech detection," in *Proceedings of the 2021 Edition of the Automatic Speaker Verification and Spoofing Countermeasures Challenge*, 2021, pp. 47–54.
- [7] ASVspoof Consortium, "Asvspoof 2021 challenge: Speech deepfake database," Zenodo, 2021. [Online]. Available: <https://doi.org/10.5281/zenodo.4835108>
- [8] X. Liu, X. Wang, M. Sahidullah, J. Patino, H. Delgado, T. Kinnunen, M. Todisco, J. Yamagishi, N. Evans, A. Nautsch, and K. A. Lee, "Asvspoof 2021: Towards spoofed and deepfake speech detection in the wild," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 31, pp. 2507–2522, 2023.
- [9] Y. Li, M. Zhang, M. Ren, X. Qiao, M. Ma, D. Wei, and H. Yang, "Cross-domain audio deepfake detection: Dataset and analysis," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Miami, Florida, USA: Association for Computational Linguistics, 2024, pp. 4977–4983. [Online]. Available: <https://aclanthology.org/2024.emnlp-main.286/>
- [10] O. C. Phukan, G. S. Kashyap, A. B. Buduru, and R. Sharma, "Heterogeneity over homogeneity: Investigating multilingual speech pre-trained models for detecting audio deepfake," 2024. [Online]. Available: <https://arxiv.org/abs/2404.00809>
- [11] A. Baevski, H. Zhou, A. Mohamed, and M. Auli, "wav2vec 2.0: A framework for self-supervised learning of speech representations," *Advances in Neural Information Processing Systems*, vol. 33, pp. 12 449–12 460, 2020. [Online]. Available: <https://proceedings.neurips.cc/paper/2020/hash/92d1e1eb1cd6f9fba3227870bb6d7f07-Abstract.html>
- [12] T. Kang, S. Han, S. Choi, J. Seo, S. Chung, S. Lee, S. Oh, and I.-Y. Kwak, "Experimental study: Enhancing voice spoofing detection models with wav2vec 2.0," 2024. [Online]. Available: <https://arxiv.org/abs/2402.17127>
- [13] X. Meng, J. Bradley, B. Yavuz, E. Sparks, S. Venkataraman, D. Liu, J. Freeman, D. B. Tsai, M. Amde, S. Owen, D. Xin, R. Xin, M. J. Franklin, R. Zadeh, M. Zaharia, and A. Talwalkar, "Mllib: Machine learning in apache spark," *Journal of Machine Learning Research*, vol. 17, no. 34, pp. 1–7, 2016. [Online]. Available: <https://www.jmlr.org/papers/v17/15-237.html>
- [14] M. Armbrust, A. Ghodsi, R. Xin, and M. Zaharia, "Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics," in *11th Annual Conference on Innovative Data Systems Research (CIDR 2021)*, 2021. [Online]. Available: https://www.cidrdb.org/cidr2021/papers/cidr2021_paper17.pdf
- [15] Databricks, "Databricks free edition limitations," Databricks Documentation, 2026, consultado en junio de 2026. [Online]. Available: <https://docs.databricks.com/aws/en/getting-started/free-edition-limitations>
- [16] B. Krawczyk, "Learning from imbalanced data: Open challenges and future directions," *Progress in Artificial Intelligence*, vol. 5, pp. 221–232, 2016.
- [17] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the roc plot when evaluating binary classifiers on imbalanced datasets," *PLOS ONE*, vol. 10, no. 3, p. e0118432, 2015.
- [18] H. Delgado, N. Evans, T. Kinnunen, K. A. Lee, X. Liu, A. Nautsch, J. Patino, M. Sahidullah, M. Todisco, X. Wang, and J. Yamagishi, "Asvspoof 2021: Automatic speaker verification spoofing and countermeasures challenge evaluation plan," 2021. [Online]. Available: <https://arxiv.org/abs/2109.00535>
- [19] D. R. Cox, "The regression analysis of binary sequences," *Journal of the Royal Statistical Society: Series B*, vol. 20, no. 2, pp. 215–242, 1958.
- [20] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [21] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.